

CSE332: Computer Architecture and Organization
Assignment#4
Spring 2025

Name: _____

Student ID: _____

Direction: You can consult any resources such as books, online references, and videos for this assignment, however, you have to properly cite and paraphrase your answers when it is necessary. There will be points for partial attempts. You can upload a typed or handwritten copy of your assignment to the canvas. However, you have to show the original copy of your assignment in case of arising questions about the authenticity. You should upload your assignment to canvas.

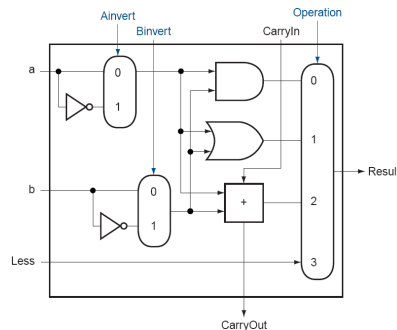
Submission guidelines: All our assignments will be based on 100 points so that we can assign weights to get your points for final grade. Whoever fails to submit within the time period assigned through NSU canvas but submit the assignment within the next three days will be punished by 20% deduction of points (e.g. start with -20 points within 100 points). The late submission will be open until semester's end through email to TA and CC to the instructor. After the first 3 days of grace period, the deduction will be 30%

Note: You are allowed to use only instructions implemented by the actual MIPS hardware provided in the Canvas references. Use assembly language format from the references or the book. Note, base ten numbers are listed as normal (e.g. 23), binary numbers are prefixed with 0b or format such as XX_2 and hexadecimal numbers are prefixed with 0x or format such as XX_{16} / XX_{HEX}

Note: If you have handwritten your assignment, you have to submit a scanned copy of it. Actions against plagiarism will be strictly enforced and proper action will be taken.

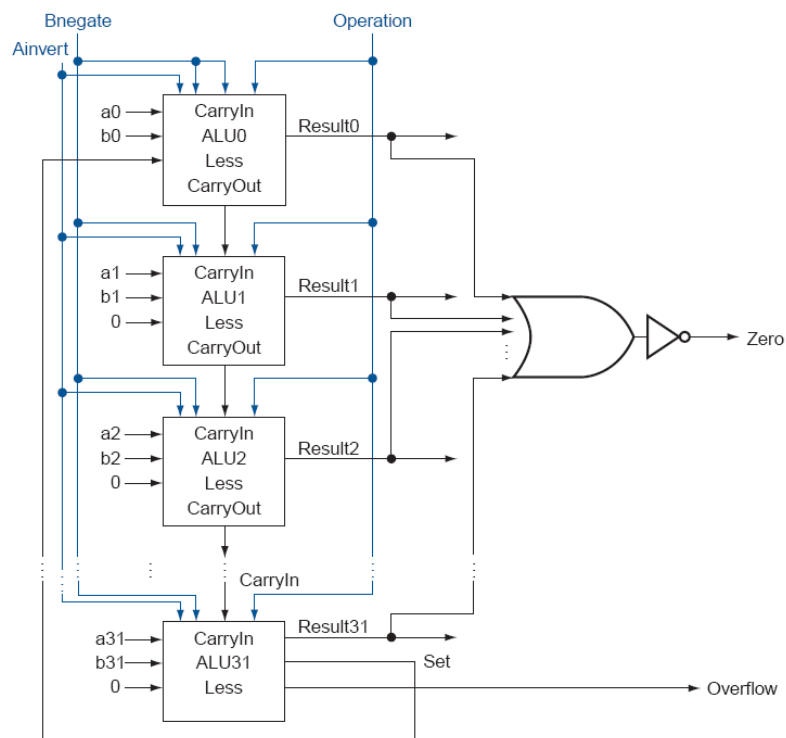
1. **(100 points=25x4)** Design an 32 bit ALU in Verilog with OR, AND, NOR, ADD, SUB, SLT logic as shown in the figure based on the given models. Attach screen-shots of outputs, answers to the questions, verilog code. Moreover, submit your code as separate files so that the grader can test it. **Check the tutorial at the end.**
 - a. Write your verilog program for the 32 bit ALU
 - b. Discuss what type of Verilog models (e.g. Structural, Dataflow, hierarchical etc.) you have used and why? Also show your comments for all the parameters such as input and output ports, connections, instances of other lower level modules etc.
 - c. Write a testbench for the 32 bit ALU. Use the tutorial (at the end) for writing your testbench. In the case of random ALUcontrol, use only five control signals that can be tested
 - d. Compile and run your simulations in ModelSim. Test whether your results are correct.

Hint: You have to add Ainvert, Binvert and Less in your enhanced full bit adder, your final Op should be a 4 bit input signal. Op[0] will be Ainvert, Op[1] will be Bnegate, Op[2:3] will be selection input to the Mux. We can also detect the overflow by checking whether the CarryIn and CarryOut of the most significant bit are different, such as xoring as C30 and Cout (e.g. assign Ov=C30^C31;). Taking xor of overflow and Result of the most significant bit can give us LessThan (assign Set = Ov ^ Result[31]) which should be connected to LESS input of the least significant bit of 32 bit ALU. An instance of 1-bit ALU may look like `full_adder_en FA2(A[1], B[1], 1'b0, C0, Op[0], Op[1], Op[2:3], Result[1], C1);`. And Set can replace 1'b0 in LT position in case of the first 1-bit ALU. Moreover, the last ALU will have a different connection than the other ALUs to support SLT operation (See the lecture).



`full_adder_en FA1(A[0], B[0], Set, Op[0], Op[1], Op[1], Op[2:3], Result[0], C0);`

`full_adder_en FA2(A[1], B[1], 1'b0, Op[0], Op[1], C0, Op[2:3], Result[1], C1);`



```

module alu16 (A,B,Cin,Op, Result, Cout);
    input [15:0] A;
    input [15:0] B;
    input Cin, Op;
    output [15:0] Result;
    output Cout;
    //wire[1:16] C; //C[1], C[2]......
    wire C0, C1, C2, C3, C4, C5, C6, C7,C8, C9, C10, C11, C12, C13, C14, C15;

    full_adder_en FA1(A[0], B[0], OP[1], Op, Result[0], C0); //C[1]
    full_adder_en FA2(A[1], B[1], C0,Op,Result[1], C1); //C[2]
    full_adder_en FA3(A[2], B[2], C1,Op, Result[2], C2);
    full_adder_en FA4(A[3], B[3], C2,Op, Result[3], C3);
    full_adder_en FA5(A[4], B[4], C3,Op, Result[4], C4); //C[5]
    full_adder_en FA6(A[5], B[5], C4,Op, Result[5], C5); //C[6]
    full_adder_en FA7(A[6], B[6], C5,Op, Result[6], C6);
    full_adder_en FA8(A[7], B[7], C6,Op, Result[7], C7);
    full_adder_en FA9(A[8], B[8], C7,Op, Result[8], C8); //C[9]
    full_adder_en FA10(A[9], B[9], C8,Op, Result[9], C9); //C[10]
    full_adder_en FA11(A[10], B[10], C9,Op, Result[10], C10);
    full_adder_en FA12(A[11], B[11], C10,Op, Result[11], C11);
    full_adder_en FA13(A[12], B[12], C11,Op, Result[12], C12); //C[13]
    full_adder_en FA14(A[13], B[13], C12,Op, Result[13], C13); //C[14]
    full_adder_en FA15(A[14], B[14], C13,Op, Result[14], C14); //C[15]
    full_adder_en FA16(A[15], B[15], C14,Op, Result[15], Cout); //C[16]
    //overflow logic
    //set logic

```

endmodule

**//the following code is not complete. Modify and complete the following enhanced
//full bit adder according your need**

```

module full_adder_en (A,B,Cin, Ainvert, Binvert, Op, Result Cout);
    input A,B,Cin;
    input Ainvert, Binvert,
    // input Op[0:3];
    output Out, Cout
    wire Sum, andX;;

    //assign ainv =~A;
    //mux2x1 m1(A, ainv, Ainvert, N1)

    //assign Sum = (A & B & Cin) | (~A & ~B & Cin) | (~A & B & ~Cin) | (A & ~B & ~Cin);

```

```

//assign Cout = (A & Cin) | (A & B) | (B & Cin);
//assign out[0] = N1&N2;
//assign out1] = N1|N2;

// //mux4x1 m2(Sum, andX, orX, Less, Result, Out)

//assign andX = A & B;
//mux2x1 m1(Sum, andX, S, Result)
//mux4x1 m2(Sum, andX, orX, Less, Result, Out)

```

endmodule

```

module mux4x1 (A, B, C, D, Sel, Q);
    input A, B, C, D;
    input [1:0] Sel;
    output reg Q;
    always @(*) // * means all the variables
        if (Sel==2'b00)
            Q = A;
        else if (Sel == 2'b01)
            Q = B;
        else if (Sel == 2'b10)
            Q = C;
        else
            Q = D;

```

endmodule

```

module mux2x1 (A, B, Sel, Q);
    input A, B, Sel;
    output Q
    reg Q;
    always @(*) // * means all the variables
        if (Sel==1'b0)
            Q = A;
        else if (Sel == 1'b1)
            Q = B;

```

endmodule

Instructions about the Verilog Simulator

- You can use Verilog Online Simulator such as Jdoodle <https://www.jdoodle.com/execute-verilog-online> for small and simple program
- However, to run a CPU model, you need to install advanced Simulator such as ModelSim
- You may download and install ModelSim by first going to page: <https://www.intel.com/content/www/us/en/software-kit/750511/modelsim-intel-fpgas-pro-edition-software-version-20-1.html>
- Download both parts and install it. It is free software and no license is required!
- To run a ModelSim simulation, you may
 - Create a new project with a name
 - Then a window will appear where a new file can be created
 - Create a file with Verilog extension using arbitrary name
 - Write a Verilog program or copy and paste a program. You can create two files, name them as **four_bit_adder.v** and **test fb_adder.v**, **copy and paste the code from the following sections.**
 - In the menu bar, click “Compile.” In the drop-down menu, click “Compile.” A dialogue window should pop-up. Select the file to compile and click the “Compile” button. If there is no problem with your code, the compile should pass, and then you should click the “Done” button to close the dialogue window.
 - In the menu bar, click “Simulate.” In the drop-down menu, click “Start Simulation.” A dialogue window should pop-up. There should be a list showing up in the window. Click on the “+” sign on “work” which is the first item in the list. Click on “YOUR_TESTBENCH.V” Then click the “OK” button. The dialogue should disappear.
 - After a little while, left click on “YOUR_TESTBENCH.V” in the “workspace” window to select it, then right click. In the menu, select “Add Wave” to add all the parameters you want to check
 - There will be a new window waveform popping up. First, find the box showing “100 ps” and change it to “1000 ps.” Then click the run simulation sign right next to the box. The waveform should be ready!

Once you install the Simulator, test the following verilog program:

//four_bit_adder.v

```
module four_bit_adder (A,B,Cin,Sum, Cout);
```

```
    input [3:0] A;
```

```
    input [3:0] B;
```

```
    input Cin;
```

```
    output [3:0] Sum;
```

```
    output Cout;
```

```
    wire C0, C1, C2;
```

```
    full_adder FA1(A[0], B[0], Cin, Sum[0], C0);
```

```
    full_adder FA2(A[1], B[1], C0, Sum[1], C1);
```

```
    full_adder FA3(A[2], B[2], C1, Sum[2], C2);
```

```
    full_adder FA4(A[3], B[3], C2, Sum[3], Cout);
```

```
endmodule
```

```
module full_adder (A,B,Cin,Sum, Cout);
```

```
    input A,B,Cin;
```

```
    output Sum, Cout;
```

```
    assign Sum = (A & B & Cin) | (~A & ~B & Cin) | (~A & B & ~Cin) | (A & ~B & ~Cin);
```

```
    assign Cout = (A & Cin) | (A & B) | (B & Cin);
```

```
endmodule
```

```

//test fb_adder.v
module test fb_adder;
// Declare testbench variables
    reg [3:0] a;
    reg [3:0] b;
    reg c_in;
    wire [3:0] sum;
    integer i;
    four_bit_adder fa0 ( .A (a), .B (b), .Cin (c_in), .Cout (c_out), .Sum (sum));
// 3. Provide stimulus to test the design
    initial begin
        a <= 0;
        b <= 0;
        c_in <= 0;
        $monitor ("a=0x%0h b=0x%0h c_in=0x%0h c_out=0x%0h sum=0x%0h", a, b, c_in,
c_out, sum);
        // Use a for loop to apply random values to the input
        for (i = 0; i < 1000; i = i+1) begin
            #10 a <= $random;
            b <= $random;
            c_in <= $random;
        end
    end
endmodule

```